

ADP470S

Applications Development Practice 4

Practice Tests — Model Solutions

Cape Peninsula University of Technology | 2026

Papers covered in this document

- Paper 1 — ADP470S Practice Questions (unweighted practice set)
- Paper 2 — ADP470S April Test 2025 (75 marks, 3 April 2025)
- Paper 3 — ADP470S Test 1 2021 (80 marks, 10 May 2021)

All solutions include step-by-step workings, fully annotated code and complexity analysis so you can understand the method, not just the answer.

PAPER 1 — ADP470S Practice Questions

Paper overview

This practice set covers: recursive & iterative factorial, Big O analysis of code excerpts, Bubble Sort trace, Insertion Sort trace, and Quick Sort trace with analysis.

Q1 Implement a recursive method for factorial

[5 marks]

A recursive method has two parts: a base case (stopping condition) and a recursive case that calls itself with a smaller input.

```
// Recursive Factorial - Java
public static int factorialRecursive(int n) {
    if (n == 0 || n == 1) {    // base case: 0! = 1 and 1! = 1
        return 1;
    }
    return n * factorialRecursive(n - 1); // recursive case
}

// Python equivalent
def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial_recursive(n - 1)
```

How the recursion unfolds for n=4

```
factorial(4) → 4 × factorial(3)
factorial(3) → 3 × factorial(2)
factorial(2) → 2 × factorial(1)
factorial(1) → 1           ← base case reached
Unwinding: 2×1=2, 3×2=6, 4×6=24 ✓
```

Q2 Implement an iterative method for factorial

[5 marks]

The iterative version uses a loop instead of calling itself. No call stack frames are created beyond the initial call.

```
// Iterative Factorial - Java
public static int factorialIterative(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result = result * i;
    }
    return result;
}

// Python equivalent
def factorial_iterative(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

Iteration trace for n=4

```

Start: result=1
i=1: result = 1 × 1 = 1
i=2: result = 1 × 2 = 2
i=3: result = 2 × 3 = 6
i=4: result = 6 × 4 = 24  ✓

```

Q3 Using the concept of a stack frame, explain the performance of the recursive method using Big O notation

[10 marks]

Stack Frames and the Call Stack

Every time a function is called, the runtime creates a new stack frame and pushes it onto the call stack. Each frame stores:

- The value of the parameter n for that particular call
- The local variables of that call
- The return address — where execution must resume after the call returns

For factorial(n), the call stack at maximum depth looks like this:

Call stack at deepest point — factorial(5)

factorial(0) n=0	← TOP OF STACK (grows upward) ← BASE CASE — returns 1
factorial(1) n=1	← waiting for factorial(0)
factorial(2) n=2	← waiting for factorial(1)
factorial(3) n=3	← waiting for factorial(2)
factorial(4) n=4	← waiting for factorial(3)
factorial(5) n=5	← first call (bottom of stack)

Time Complexity — $O(n)$

For factorial(n), the function calls itself exactly n times ($n, n-1, n-2, \dots, 1, 0$). Each call does exactly one multiplication (or returns 1 at the base case). The total number of operations is proportional to n , therefore:

Time Complexity of Recursive Factorial = $O(n)$

As n doubles, the number of function calls doubles. The relationship is linear.

Space Complexity — $O(n)$

Each of the n recursive calls occupies its own stack frame simultaneously. At the deepest point, all $n+1$ frames (factorial(0) through factorial(n)) are live on the call stack at the same time. Memory used grows linearly with n :

Space Complexity of Recursive Factorial = $O(n)$

n stack frames are allocated simultaneously. For very large n , this risks a StackOverflow error.

Q4 Compare the performance of recursive vs iterative factorial. Which is preferable in the real world?

[5 marks]

Metric	Recursive	Iterative	Comment
Time Complexity	$O(n)$	$O(n)$	Equal — both perform n multiplications
Space Complexity	$O(n)$	$O(1)$	Iterative wins — no stack frames
Stack Overflow Risk	Yes (large n)	No	Iterative is safer
Code Clarity	High	Moderate	Recursive mirrors the math formula
Real-World Preference	—	✓ Preferred	Iterative for production code

Conclusion

The iterative method is preferable in real-world software because it uses constant $O(1)$ space — no matter how large n is, only a single result variable is maintained. The recursive version, while elegant and mathematically clear, allocates n stack frames simultaneously, risking a StackOverflow for large inputs and consuming significantly more memory.

Q5a Determine the Big O expression for the code excerpt (two sequential loops)

[3 marks]

```
int sum = 0;
for (int i = 0; i < n; i++) {
    sum = sum + a[i];           // Loop A: runs n times
}
for (int j = 0; j < n; j++) {
    sum = sum + a[j];           // Loop B: runs n times
}
```

Analysis:

- Loop A executes n iterations — $O(n)$
- Loop B executes n iterations — $O(n)$
- The loops are sequential (one after the other), so total work = $n + n = 2n$

- In Big O notation we drop the constant coefficient 2

Answer: $O(n) + O(n) = O(2n) = O(n)$

Q5b Determine the Big O expression for the code excerpt (nested loops, $j = j * 3$)

[3 marks]

```
int prod = 1;
for (int i = 1; i < n; i++) {           // Outer: runs n times
    for (int j = 1; j < n; j = j * 3) { // Inner: j = 1, 3, 9, 27, ... → log3(n)
        times
        prod = prod * j;
    }
}
```

Analysis:

- Outer loop: i increments by 1 from 1 to $n \rightarrow$ runs n times
- Inner loop: j starts at 1 and is multiplied by 3 each iteration ($j = 1, 3, 9, 27, 81, \dots$)
- The inner loop terminates when $j \geq n$. Since $j = 3^k$, we need $3^k = n \rightarrow k = \log_3(n)$
- So the inner loop runs $\log_3(n) \approx \log(n)$ times for each outer iteration
- Total operations = $n \times \log(n)$

Answer: $O(n \log n)$

Q5c Determine the Big O expression for the code excerpt (outer $i+=3$, inner $j--$)

[3 marks]

```
int sum = 0;
for (i = 0; i <= n-1; i += 3) {        // Outer: steps of 3 → runs n/3 times
    for (j = n-1; j > 0; j--) {        // Inner: j from n-1 down to 1 → runs n-1
        times
        sum = sum + (i * j);
    }
}
```

Analysis:

- Outer loop: i goes 0, 3, 6, 9, ..., $n-1 \rightarrow$ runs approximately $n/3$ times
- Inner loop: j goes from $n-1$ down to 1 $\rightarrow$ runs $n-1 \approx n$ times
- Total iterations = $(n/3) \times n = n^2/3$
- In Big O notation we drop the constant $1/3$

Answer: $O(n/3 \times n) = O(n^2/3) = O(n^2)$

Q6 Show how Bubble Sort modifies the array {90, 23, 5, 109, 12}**[12 marks]**

Bubble Sort compares adjacent elements and swaps them if the left is greater than the right. After each pass, the largest unsorted element is in its correct position.

Pass 1 — bubbling the maximum (109) to the end

Step	Array State	Action
Start	[90, 23, 5, 109, 12]	Initial array
Compare a[0]=90, a[1]=23	[23, 90, 5, 109, 12]	90 > 23 → SWAP ↑
Compare a[1]=90, a[2]=5	[23, 5, 90, 109, 12]	90 > 5 → SWAP ↑
Compare a[2]=90, a[3]=109	[23, 5, 90, 109, 12]	90 < 109 → no swap
Compare a[3]=109, a[4]=12	[23, 5, 90, 12, 109]	109 > 12 → SWAP ↑ → 109 in place ✓

Pass 2 — bubbling 90 to its position

Step	Array State	Action
Start	[23, 5, 90, 12, 109]	109 already in place
Compare a[0]=23, a[1]=5	[5, 23, 90, 12, 109]	23 > 5 → SWAP ↑
Compare a[1]=23, a[2]=90	[5, 23, 90, 12, 109]	23 < 90 → no swap
Compare a[2]=90, a[3]=12	[5, 23, 12, 90, 109]	90 > 12 → SWAP ↑ → 90, 109 in place ✓

Pass 3 — bubbling 23 to its position

Step	Array State	Action
Start	[5, 23, 12, 90, 109]	90, 109 already in place
Compare a[0]=5, a[1]=23	[5, 23, 12, 90, 109]	5 < 23 → no swap
Compare a[1]=23, a[2]=12	[5, 12, 23, 90, 109]	23 > 12 → SWAP ↑ → 23, 90, 109 in place ✓

Pass 4 — final pass (confirm sorted)

Step	Array State	Action
Compare a[0]=5, a[1]=12	[5, 12, 23, 90, 109]	5 < 12 → no swap
Result	[5, 12, 23, 90, 109]	SORTED ✓

Big O of Bubble Sort

- Worst case (reverse sorted): $O(n^2)$ — $(n-1)$ passes $\times$ $(n-1)$ comparisons each
- Best case (already sorted, with early exit): $O(n)$ — only 1 pass needed
- Average case: $O(n^2)$ — random input requires roughly $n^2/2$ comparisons

Q7 Show how Insertion Sort modifies the array {90, 23, 5, 109, 12}**[12 marks]**

Insertion Sort builds a sorted region on the left. For each element, it is inserted into the correct position within the already-sorted left region by shifting larger elements right.

Pass	Key value	Action	Array after insertion
i=1	key=23	23 < 90 → shift 90 right; insert 23 at index 0	[23, 90, 5, 109, 12]
i=2	key=5	5 < 90 → shift; 5 < 23 → shift; insert 5 at index 0	[5, 23, 90, 109, 12]
i=3	key=109	109 > 90 → no shift; insert 109 at index 3	[5, 23, 90, 109, 12]
i=4	key=12	12 < 109 → shift; 12 < 90 → shift; 12 < 23 → shift; 12 > 5 → insert at index 1	[5, 12, 23, 90, 109] ✓

Detailed trace for i=4 (key=12):

```
Sorted region: [5, 23, 90, 109]   key = 12
j=3: a[3]=109 > 12 → a[4]=a[3]   → array: [5, 23, 90, 109, 109]
j=2: a[2]=90  > 12 → a[3]=a[2]   → array: [5, 23, 90, 90, 109]
j=1: a[1]=23  > 12 → a[2]=a[1]   → array: [5, 23, 23, 90, 109]
j=0: a[0]=5   < 12 → STOP
Place key: a[j+1] = a[1] = 12   → array: [5, 12, 23, 90, 109] ✓
```

Big O of Insertion Sort

- Best case (already sorted): $O(n)$ — each element just compared once with its predecessor
- Worst case (reverse sorted): $O(n^2)$ — maximum shifts on every insertion
- Average case: $O(n^2)$
- Advantage over Bubble Sort: fewer total swaps; moves elements in single shift operations

Q8 Show Quick Sort on {9, 7, 5, 11, 12, 2, 14, 3, 10, 6} — always choose the middle element as pivot

[12 marks]

Quick Sort uses divide-and-conquer: choose a pivot, partition the array so that all elements less than the pivot are to its left and all greater are to its right, then recursively sort each partition.

Level 1 — Full array, pivot = $a[4] = 12$

Array: [9, 7, 5, 11, 12, 2, 14, 3, 10, 6] (indices 0–9, mid index = 4, pivot = 12)

Partition: move elements < 12 to left, elements > 12 to right:

Partition trace — pivot = 12

Initial: [9, 7, 5, 11, 12, 2, 14, 3, 10, 6]

$i=0, j=9$: $a[0]=9 < 12 \rightarrow i++$; $a[1]=7 < 12 \rightarrow i++$; $a[2]=5 < 12 \rightarrow i++$; $a[3]=11 < 12 \rightarrow i++$; $a[4]=12$ stop ($i=4$)

$a[9]=6$ not $> 12 \rightarrow j$ stops at 9

$i=4 \leq j=9 \rightarrow \text{swap}(a[4], a[9]) \rightarrow [9, 7, 5, 11, 6, 2, 14, 3, 10, 12]$ $i=5, j=8$

$a[5]=2 < 12 \rightarrow i++$; $a[6]=14$ stop ($i=6$)

$a[8]=10$ not $> 12 \rightarrow j$ stops at 8

$i=6 \leq j=8 \rightarrow \text{swap}(a[6], a[8]) \rightarrow [9, 7, 5, 11, 6, 2, 10, 3, 14, 12]$ $i=7, j=7$

$a[7]=3 < 12 \rightarrow i++$; $i=8 > j=7 \rightarrow$ done. Partition returns $i=8$

After L1: LEFT=[9, 7, 5, 11, 6, 2, 10, 3] (idx 0–7) RIGHT=[14, 12] (idx 8–9)

Level 2a — Sort [9, 7, 5, 11, 6, 2, 10, 3] — pivot = $a[3] = 11$

Partition trace — pivot = 11

Initial subarray (idx 0–7): [9, 7, 5, 11, 6, 2, 10, 3]

mid = $(0+7)/2 = 3 \rightarrow$ pivot = $a[3] = 11$

Scan: $a[0]=9 < 11 \rightarrow i++$; $a[1]=7 < 11 \rightarrow i++$; $a[2]=5 < 11 \rightarrow i++$; $a[3]=11$ stop ($i=3$)

$a[7]=3$ not $> 11 \rightarrow j$ stays at 7

$i=3 \leq j=7 \rightarrow \text{swap}(a[3], a[7]) \rightarrow [9, 7, 5, 3, 6, 2, 10, 11]$ $i=4, j=6$

$a[4]=6 < 11 \rightarrow i++$; $a[5]=2 < 11 \rightarrow i++$; $a[6]=10 < 11 \rightarrow i++$; $i=7 > j=6 \rightarrow$ done. Returns

$i=7$

After L2a: LEFT=[9, 7, 5, 3, 6, 2, 10] (idx 0–6) RIGHT=[11] (idx 7 – in place ✓)

Level 2b — Sort [14, 12] — pivot = $a[8] = 14$ (indices 8–9)

Partition trace — pivot = 14

Subarray (idx 8–9): [14, 12] mid = $(8+9)/2=8$ pivot = $a[8]=14$

$a[8]=14$ not $< 14 \rightarrow i$ stays at 8

$a[9]=12$ not $> 14 \rightarrow j$ stays at 9

$i=8 \leq j=9 \rightarrow \text{swap}(a[8], a[9]) \rightarrow [12, 14]$ $i=9, j=8 \rightarrow$ done. Returns $i=9$

After L2b: [12, 14] sorted ✓

Levels 3 onwards — sorting [9, 7, 5, 3, 6, 2, 10]

Continuing recursively on [9, 7, 5, 3, 6, 2, 10] — mid index = 3, pivot = $a[3] = 3$:

Partition trace — pivot = 3 on subarray [9, 7, 5, 3, 6, 2, 10]


```

a[0]=9 not<3 → i stops at 0
a[6]=10>3→j--; a[5]=2 not>3 → j=5
swap(a[0],a[5]) → [2,7,5,3,6,9,10] i=1,j=4
a[1]=7 not<3 → i=1; a[4]=6>3→j--; a[3]=3 not>3 → j=3
swap(a[1],a[3]) → [2,3,5,7,6,9,10] i=2,j=2
a[2]=5>3 → i stays at 2; a[2]=5>3→j--; j=1 → i=2>j=1 → done. Returns 2
LEFT=[2] RIGHT=[3,5,7,6,9,10] (continuing...)

```

Final sorted result

After all recursive partitioning completes:

[2, 3, 5, 6, 7, 9, 10, 11, 12, 14] ✓

Q9 Explain the performance of Quick Sort: best case, average case, worst case

[6 marks]

Case	Complexity	Explanation
Best Case	$O(n \log n)$	The pivot always divides the array into two equal halves. The recursion tree has depth $\log_2(n)$ and at each level, all n elements are processed once. Total = $n \times \log(n)$.
Average Case	$O(n \log n)$	With a randomly chosen pivot, the partition produces reasonably balanced halves on average. Analysis shows the expected number of comparisons is approximately $1.39 \times n \log n$.
Worst Case	$O(n^2)$	Occurs when the pivot is always the smallest or largest element (e.g. already-sorted array with first/last element as pivot). The recursion tree has depth n , and total work is $n + (n-1) + \dots + 1 = n^2/2$.

Space complexity

Quick Sort uses $O(\log n)$ space on average for the recursion call stack (depth of balanced recursion tree). In the worst case, stack depth is $O(n)$.

PAPER 2 — ADP470S April Test 2025 (75 Marks, 3 April 2025)

Paper overview

This is the most recent test paper. It covers Big O analysis, recursive factorial with stack trace, and sorting traces + complexity for Bubble Sort, Quick Sort, and Heap Sort on the array {90, 23, 5, 109, 12}.

Section 1 — Big O Expressions

(These are the same three code excerpts as Practice Q5. Full workings are shown again below.)

Q1.1 Big O — Two sequential loops

[3 marks]

```
int sum = 0;
for (int i = 0; i < n; i++) { sum = sum + a[i]; } // O(n)
for (int j = 0; j < n; j++) { sum = sum + a[j]; } // O(n)
```

$$O(n) + O(n) = O(2n) = O(n)$$

Two sequential independent loops both linear in n . Constants are dropped in Big O.

Q1.2 Big O — Nested loops (outer $i++$, inner j starts at i , $j = j * 3$)

[3 marks]

```
int prod = 1;
for (int i = 1; i < n; i++) { // outer: O(n)
    for (int j = i; j < n; j = j*3) { // inner: j = i, 3i, 9i, ... → log3(n/i)
        times
        prod = prod * j;
    }
}
```

Analysis: The outer loop runs n times. The inner loop starts at $j=i$ and triples j each step. The inner loop runs approximately $\log_3(n)$ times regardless of i (since for small i , j quickly reaches n). The total is $O(n \log n)$.

$$O(n \log n)$$

Q1.3 Big O — Nested loops (outer $i+=3$, inner $j--$)

[3 marks]

```
int sum = 0;
for (i=0; i<=n-1; i+=3) { // outer: runs n/3 times
    for (j=n-1; j>0; j--) { // inner: runs n-1 ≈ n times
        sum = sum + (i * j);
    }
}
```

$$O((n/3) \times n) = O(n^2/3) = O(n^2)$$

The constant $1/3$ is absorbed by Big O notation. Both loops are linearly proportional to n .

Section 2 — Recursive Factorial

Q2.1 Demonstrate execution of recursive factorial and report time and space complexity

[marks]

Recursive definition:

$$n! = \begin{cases} 1 & \text{if } n = 0 \text{ or } n = 1 \\ n \times \text{factorial}(n-1) & \text{if } n > 1 \end{cases}$$

```
public static int factorial(int n) {
    if (n == 0 || n == 1) return 1;    // base case
    return n * factorial(n - 1);      // recursive call
}
```

Execution trace for factorial(5)

Call stack — expanding (each new call pushes a frame)

```
CALL factorial(5) → needs factorial(4)
CALL factorial(4) → needs factorial(3)
CALL factorial(3) → needs factorial(2)
CALL factorial(2) → needs factorial(1)
CALL factorial(1) → returns 1 ← BASE CASE
```

Call stack — unwinding (each RETURN pops a frame)

```
RETURN factorial(2) = 2 × 1 = 2
RETURN factorial(3) = 3 × 2 = 6
RETURN factorial(4) = 4 × 6 = 24
RETURN factorial(5) = 5 × 24 = 120 ✓
```

Q2.2 Report space complexity and time complexity in Big O

[marks]

Complexity	Big O	Justification
Time Complexity	$O(n)$	The function calls itself exactly n times. Each call does a constant amount of work (one multiplication). Total work scales linearly with n .
Space Complexity	$O(n)$	Each recursive call allocates a stack frame. At peak depth, n frames are live simultaneously on the call stack — one for each call from factorial(n) down to factorial(0).

Section 3 — Sorting Algorithms on {90, 23, 5, 109, 12}**Q3.1** Show how Bubble Sort modifies the array {90, 23, 5, 109, 12}**[marks]**

(Full trace — identical to Practice Q6.)

Pass	Array after pass	Largest element placed
Initial	[90, 23, 5, 109, 12]	—
Pass 1	[23, 5, 90, 12, 109]	109 ✓
Pass 2	[5, 23, 12, 90, 109]	90 ✓
Pass 3	[5, 12, 23, 90, 109]	23 ✓
Pass 4	[5, 12, 23, 90, 109]	Confirmed sorted ✓

Q3.2 Show and explain the Big O expression for Bubble Sort**[marks]**

Bubble Sort has two nested loops:

- Outer loop: runs $n-1$ passes (one per unsorted element)
- Inner loop: runs $n-1-j$ comparisons on pass j (shrinks each pass)
- Total comparisons = $(n-1) + (n-2) + \dots + 1 = n(n-1)/2 \approx n^2/2$

Big O of Bubble SortWorst case (reverse sorted): $O(n^2)$ comparisons, $O(n^2)$ swapsBest case (already sorted): $O(n)$ with early-exit optimisationAverage case: $O(n^2)$ Space complexity: $O(1)$ — in-place, only a temp variable needed**Q3.3** Show how Quick Sort modifies the array {90, 23, 5, 109, 12}**[marks]**

Using Hoare's partition with middle element as pivot. Array: [90, 23, 5, 109, 12]

Level 1 — pivot = $a[(0+4)/2] = a[2] = 5$ **Partition around pivot 5**

Initial: [90, 23, 5, 109, 12] pivot=5

 $i=0$: $a[0]=90 \text{ not } < 5 \rightarrow \text{stop at } i=0$ $j=4$: $a[4]=12 > 5 \rightarrow j=3$; $a[3]=109 > 5 \rightarrow j=2$; $a[2]=5 \text{ not } > 5 \rightarrow \text{stop at } j=2$ $i=0 \leq j=2 \rightarrow \text{swap}(a[0], a[2]) \rightarrow [5, 23, 90, 109, 12]$ $i=1, j=1$ $i=1$: $a[1]=23 \text{ not } < 5 \rightarrow \text{stop}$; $j=1$: $a[1]=23 > 5 \rightarrow j=0$; $a[0]=5 \text{ not } > 5 \rightarrow \text{stop at } j=0$ $i=1 > j=0 \rightarrow \text{partition done. Returns } i=1$ After: [5 | 23, 90, 109, 12] $\rightarrow$ 5 is in correct position ✓**Level 2 — Sort [23, 90, 109, 12] (indices 1–4), pivot = $a[(1+4)/2] = a[2] = 90$**

Partition around pivot 90

```
Subarray (idx 1-4): [23, 90, 109, 12]  pivot=90
i=1: 23<90→i++; i=2: 90 not<90 → stop (i=2)
j=4: 12 not>90 → stop (j=4)
swap(a[2],a[4]) → [5, 23, 12, 109, 90]  i=3, j=3
i=3: 109 not<90 → stop; j=3: 109>90→j=2; 12 not>90 → stop (j=2)
i=3 > j=2 → done. Returns i=3
After: [5, 23, 12 | 109, 90] → subarrays: [23,12] and [109,90]
```

Level 3a — Sort [23, 12] (indices 1–2), pivot = a[1] = 23**Partition around pivot 23**

```
i=1: 23 not<23 → stop; j=2: 12 not>23 → stop
swap(a[1],a[2]) → [5, 12, 23, 109, 90]  i=2, j=1 → i>j done. Returns 2
After: [5, 12, 23 | 109, 90] → 12 and 23 now sorted ✓
```

Level 3b — Sort [109, 90] (indices 3–4), pivot = a[3] = 109**Partition around pivot 109**

```
i=3: 109 not<109 → stop; j=4: 90 not>109 → stop
swap(a[3],a[4]) → [5, 12, 23, 90, 109]  i=4, j=3 → i>j done.
Final result: [5, 12, 23, 90, 109] ✓
```

Q3.4 Show and explain the Big O expression for Quick Sort**[marks]****Big O of Quick Sort**

Best case (balanced pivots every time): $O(n \log n)$

Recursion tree depth = $\log_2(n)$; n comparisons per level → $n \log n$ total

Average case $O(n \log n)$

Random pivots produce balanced partitions on average

Worst case (pivot always min or max): $O(n^2)$

Recursion depth = n ; work per level = n → n^2 total

Space complexity (stack): $O(\log n)$ average, $O(n)$ worst case

Q3.5 Show how Heap Sort modifies the array {90, 23, 5, 109, 12}**[marks]**

Heap Sort has two phases: (1) Build a max-heap, (2) Repeatedly extract the maximum. The sift() function restores the heap property by pushing a node down to its correct position.

Phase 1 — Build Max-Heap

Start from the last non-leaf node: $\text{index} = \lfloor n/2 \rfloor - 1 = \lfloor 5/2 \rfloor - 1 = 1$

sift(a, 1) — fix node at index 1

Array: [90, 23, 5, 109, 12]

Node 1 = 23. Children: left=index 3 (109), right=index 4 (12)

Larger child = index 3 (109). Since $23 < 109 \rightarrow \text{swap}(a[1], a[3])$

$\rightarrow [90, 109, 5, 23, 12]$

sift(a, 0) — fix node at index 0 (root)

Array: [90, 109, 5, 23, 12]

Node 0 = 90. Children: left=index 1 (109), right=index 2 (5)

Larger child = index 1 (109). Since $90 < 109 \rightarrow \text{swap}(a[0], a[1])$

$\rightarrow [109, 90, 5, 23, 12]$

Now at index 1 (90): children are index 3 (23) and index 4 (12). $90 >$ both $\rightarrow$ stop.

Max-heap built: [109, 90, 5, 23, 12] ✓

Phase 2 — Extract maximum repeatedly

Step	Action	Array state
Iteration 1 (n=4)	$\text{swap}(a[0], a[4]); \text{sift}(a, 0)$ with n=4	[12, 90, 5, 23, 109] $\rightarrow$ sift $\rightarrow$ [90, 23, 5, 12, 109]
Iteration 2 (n=3)	$\text{swap}(a[0], a[3]); \text{sift}(a, 0)$ with n=3	[12, 23, 5, 90, 109] $\rightarrow$ sift $\rightarrow$ [23, 12, 5, 90, 109]
Iteration 3 (n=2)	$\text{swap}(a[0], a[2]); \text{sift}(a, 0)$ with n=2	[5, 12, 23, 90, 109] $\rightarrow$ sift $\rightarrow$ [12, 5, 23, 90, 109]
Iteration 4 (n=1)	$\text{swap}(a[0], a[1]);$ done	[5, 12, 23, 90, 109] ✓

Final result

[5, 12, 23, 90, 109] ✓

Q3.6 Show and explain the Big O expression for Heap Sort**[marks]****Big O of Heap Sort**

Build max-heap: $O(n)$ (each sift is $O(\log n)$, done for $n/2$ nodes)

Extract phase:	$O(n \log n)$	(n extractions $\times O(\log n)$ sift each)
Total:	$O(n \log n)$	in ALL cases — best, average, and worst
Space complexity:	$O(1)$	— in-place sort, no additional arrays needed

Heap Sort is preferred over Quick Sort when a guaranteed $O(n \log n)$ worst-case is required, and over Merge Sort when $O(1)$ extra space is required.

PAPER 3 — ADP470S Test 1 (80 Marks, 10 May 2021)

Paper overview

This 2021 paper covers BST implementation and traversal, circular queue, stack with linked list, and recursive/iterative factorial comparison.

Question 1 — Binary Search Tree Implementation [20 marks]

Q1.1 Create a program implementing a BST where student name is the key attribute

[20 marks]

A Binary Search Tree (BST) stores each node such that: all keys in the left subtree $<$ node key $<$ all keys in the right subtree. Student names are compared lexicographically (alphabetically).

```
// Student class
class Student {
    String name;
    String studentId;
    double overallMark;

    Student(String name, String studentId, double overallMark) {
        this.name = name;
        this.studentId = studentId;
        this.overallMark = overallMark;
    }
}

// BST Node
class BSTNode {
    Student data;
    BSTNode left, right;

    BSTNode(Student data) {
        this.data = data;
        this.left = null;
        this.right = null;
    }
}

// Binary Search Tree
class StudentBST {
    BSTNode root;

    // Insert a student (key = name, compared alphabetically)
    public void insert(Student s) {
        root = insertRec(root, s);
    }

    private BSTNode insertRec(BSTNode node, Student s) {
        if (node == null) return new BSTNode(s); // empty spot found

        int cmp = s.name.compareTo(node.data.name);
        if (cmp < 0) node.left = insertRec(node.left, s); // name <
current
        else if (cmp > 0) node.right = insertRec(node.right, s); // name >
current
        // if cmp == 0, duplicate name - skip (or handle as needed)
        return node;
    }
}
```



```
// Inorder traversal - visits nodes in alphabetical order by name
public void inorder() {
    inorderRec(root);
}

private void inorderRec(BSTNode node) {
    if (node == null) return;
    inorderRec(node.left);           // visit left subtree first
    System.out.println(node.data.name + " (" + node.data.studentId + ") - "
+ node.data.overallMark);
    inorderRec(node.right);         // then right subtree
}

// Main - demo
public class Main {
    public static void main(String[] args) {
        StudentBST tree = new StudentBST();
        tree.insert(new Student("John", "21034569", 67));
        tree.insert(new Student("Ayanda", "23245432", 98));
        tree.insert(new Student("Khaya", "214354233", 76));
        tree.insert(new Student("Zanele", "22198765", 85));
        System.out.println("Students in alphabetical order:");
        tree.inorder();
    }
}
```

Expected output (alphabetical by name)

Students in alphabetical order:

Ayanda (23245432) - 98.0

John (21034569) - 67.0

Khaya (214354233) - 76.0

Zanele (22198765) - 85.0

BST structure after inserting John, Ayanda, Khaya

```

      John          ← root (inserted first)
     /  \
    /    \
  Ayanda  Khaya    ← A < J < K alphabetically

```

Question 2 [20 marks]

Q2.1 Explain the working principle and time performance of BST insert and inorder traversal

[10 marks]

Inorder Traversal — Working Principle

Inorder traversal follows the pattern: visit LEFT subtree → visit ROOT node → visit RIGHT subtree, applied recursively to every node.

- For a correctly constructed BST this produces all nodes in ascending order (alphabetical for string keys).
- Time complexity: $O(n)$ — every node is visited exactly once, so the total work is proportional to n .
- Space complexity: $O(h)$ where h is the height of the tree. For a balanced BST $h = O(\log n)$. For a degenerate (skewed) BST $h = O(n)$.

Insert — Working Principle

To insert a new key, start at the root and repeatedly compare the new key with the current node:

- If new key < current node → go LEFT
- If new key > current node → go RIGHT
- If the current node is null (empty position) → create a new node and place it there

This maintains the BST property: left < parent < right at every node.

- Time complexity — best/average case: $O(\log n)$ for a balanced tree (height = $\log n$)
- Time complexity — worst case: $O(n)$ when the tree degenerates into a linked list (e.g. inserting sorted data without balancing)

Model answer summary (as given in paper)

Inorder: checks left child including its entire subtree, then the parent, then the right child including its entire subtree — visits each node exactly once → $O(n)$ worst case.

Insert: maintains BST rule (left < root < right) by traversing to the correct null position.

Worst case $O(n)$ if the tree is degenerate (all nodes in a single chain).

Q2.2 Explain the working principle and time performance of the circular queue

[10 marks]

A queue is a First-In-First-Out (FIFO) data structure. Elements are added at the rear (enqueue) and removed from the front (dequeue).

Problem with a standard linear queue

In a fixed-size array queue, dequeue increments the front pointer. Eventually the rear pointer reaches the end of the array even though positions at the front have been freed up — wasting space.

Circular Queue solution

A circular queue uses the modulo operator to wrap the rear pointer back to the start when it reaches the array end:

```
rear = (rear + 1) % capacity; // wraps around: after index capacity-1 comes
index 0
front = (front + 1) % capacity; // same for front pointer
```

- Enqueue: place element at rear; advance rear = $(\text{rear} + 1) \% \text{size}$
- Dequeue: remove element at front; advance front = $(\text{front} + 1) \% \text{size}$
- Full condition: $(\text{rear} + 1) \% \text{size} == \text{front}$
- Empty condition: $\text{front} == \text{rear}$

Time complexity of circular queue operations

Enqueue: $O(1)$ — direct index assignment, no loop

Dequeue: $O(1)$ — direct index read and pointer advance, no loop

Peek (front): $O(1)$

All operations are $O(1)$ because circular increment is a single arithmetic step.

Question 3 — Stack Implementation using Linked List [20 marks]**Q3.1** Implement a stack of Student objects using a linked representation**[20 marks]**

A stack is a Last-In-First-Out (LIFO) structure. Using a linked list means there is no fixed size limit. Push adds a new node at the head; pop removes the head node.

```
// Student class
class Student {
    String studentId;
    String name;
    double overallMark;

    Student(String name, String studentId, double overallMark) {
        this.name = name;
        this.studentId = studentId;
        this.overallMark = overallMark;
    }

    public String toString() {
        return name + " (" + studentId + ") - Mark: " + overallMark;
    }
}

// Stack Node
class StackNode {
    Student data;
    StackNode next;

    StackNode(Student data) {
        this.data = data;
        this.next = null;
    }
}

// Linked Stack
class LinkedStack {
    private StackNode top; // points to top element

    public boolean isEmpty() {
        return top == null;
    }

    // Push - add to top O(1)
    public void push(Student s) {
        StackNode newNode = new StackNode(s);
        newNode.next = top; // new node points to old top
        top = newNode;      // top now points to new node
        System.out.println("Pushed: " + s);
    }

    // Pop - remove from top O(1)
    public Student pop() {
        if (isEmpty()) {
            System.out.println("Stack is empty - underflow!");
            return null;
        }
        Student s = top.data;
        top = top.next; // advance top pointer
        return s;
    }

    // Peek - view top without removing O(1)
    public Student peek() {
        if (isEmpty()) return null;
    }
}
```

```

        return top.data;
    }

    // Display all elements
    public void display() {
        StackNode current = top;
        System.out.println("Stack (top to bottom):");
        while (current != null) {
            System.out.println("  " + current.data);
            current = current.next;
        }
    }
}

// Main
public class StackDemo {
    public static void main(String[] args) {
        LinkedStack stack = new LinkedStack();

        stack.push(new Student("John", "21034569", 67));
        stack.push(new Student("Ayanda", "23245432", 98));
        stack.push(new Student("Khaya", "214354233", 76));

        stack.display();

        System.out.println("\nPopping: " + stack.pop());
        stack.display();
    }
}

```

Expected output

```

Pushed: John (21034569) - Mark: 67.0
Pushed: Ayanda (23245432) - Mark: 98.0
Pushed: Khaya (214354233) - Mark: 76.0
Stack (top to bottom):
  Khaya (214354233) - Mark: 76.0
  Ayanda (23245432) - Mark: 98.0
  John (21034569) - Mark: 67.0
Popping: Khaya (214354233) - Mark: 76.0
Stack (top to bottom):
  Ayanda (23245432) - Mark: 98.0
  John (21034569) - Mark: 67.0

```

Stack structure (LIFO)

After all three pushes:

TOP → [Khaya | •] → [Ayanda | •] → [John | /]

Khaya was pushed last, so it is popped first.

Question 4 — Factorial: Recursive vs Iterative [20 marks]**Q4.1** Implement a recursive method for factorial**[5 marks]**

```
// Java - Recursive Factorial
public static int factorialRecursive(int n) {
    if (n == 0 || n == 1) {
        return 1; // base case
    }
    return n * factorialRecursive(n - 1); // recursive case
}
```

Q4.2 Implement an iterative method for factorial**[5 marks]**

```
// Java - Iterative Factorial
public static int factorialIterative(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result = result * i;
    }
    return result;
}
```

Q4.3 Using Big O notation, compare time and space utilisation. Which is preferable?**[10 marks]****Time Complexity**

Both methods compute n multiplications and therefore have the same time complexity:

Recursive Factorial Time: $O(n)$ | Iterative Factorial Time: $O(n)$

Both are linear — as n doubles, the number of operations doubles.

Space Complexity

This is where the two methods differ significantly:

Recursive — Space: $O(n)$

Each recursive call creates a new stack frame on the call stack. For $\text{factorial}(n)$, there are $n+1$ frames simultaneously active at peak depth ($n, n-1, \dots, 1, 0$). Memory usage grows linearly with n . Risk of StackOverflow for large n .

Iterative — Space: $O(1)$

Only a single integer variable (result) is used throughout the entire computation. No matter how large n is, only constant space is consumed. No stack overflow risk.

Criterion	Recursive	Iterative	Verdict
Time complexity	$O(n)$	$O(n)$	Equal — same number of multiplications

Space complexity	$O(n)$	$O(1)$	Iterative uses constant space ✓
Stack overflow risk	Yes	No	Iterative is safe for large n ✓
Code clarity	High — mirrors definition	Moderate	Recursive is easier to understand
Real-world preference	—	Preferred ✓	Iterative for production systems

Conclusion (exam-ready answer)

Both methods have identical time complexity $O(n)$. However, the iterative method is preferable in real-world applications because it operates in $O(1)$ space — no stack frames are accumulated. The recursive method's $O(n)$ space usage poses a risk of StackOverflow for large values of n . While the recursive implementation is more elegant and closer to the mathematical definition, the iterative version is the practical choice for production software where memory efficiency and safety are priorities.

— End of Solutions Document —

ADP470S Practice Tests Solutions | CPUT 2026